

Concurrent Uebungen

Euclid

$$|\mathbf{x}| = \sqrt{\sum_{k=1}^n |x_k|^2}$$

Erstelle ein **CompletableFuture**, das die **euklidische Norm** für eine Liste von **Number** berechnet.

Beispiel:

Beispiel: Gegeben ist folgende Collection: [1, 2, 3, 4, 5.5]

Berechnung: Wurzel(1*1 + 2*2 + 3*3 + 4*4 + 5.5*5.5) // *ergibt 7.762087348130012*

Tipp: Die [Math](#)-Bibliothek könnte hilfreich sein.

Das **Ergebnis** soll erst **nach zwei Sekunden zur Verfügung** stehen (Thread::sleep).

Während das CompletableFuture rechnet, sollen auf der Konsole **alle 100 ms ein Punkt "." ausgegeben** werden, bis die Berechnung des Futures fertig ist. Gebe dann das **Ergebnis des CompletableFuture auf der Konsole** aus.

ConcurrentCurrencyConverter

```
class ConcurrentCurrencyConverter {  
  
    private static final Random RANDOM = new Random();  
  
    private static double getExchangeRate(String currency) { // Simuliert einen API-Call für den Wechselkurs  
        try {  
            int millis = RANDOM.nextInt(2000) + 500;  
            Thread.sleep(millis);  
        } catch (InterruptedException e) {  
            Thread.currentThread().interrupt();  
        }  
  
        return switch (currency) { // Simulierte Wechselkurse  
            case "USD" -> 1.08;  
            case "GBP" -> 0.85;  
            case "JPY" -> 158.0;  
            default -> throw new IllegalArgumentException("Unbekannte Währung");  
        };  
    }  
  
    public static void convertAmount(double euroAmount) {  
        // TODO: Implementiere hier die parallele Umrechnung mit CompletableFuture.  
        // Wandle den Betrag 42 Euro parallel in USD, GBP und JPY um.  
        // Gib jedes Ergebnis in der Konsole aus, sobald es verfügbar ist inklusive der Berechnungszeit.  
        // Zeige am Ende die Gesamtzeit an.  
    }  
  
    public static void main(String[] args) {  
        convertAmount(42);  
    }  
}
```

Mögliche Ausgabe:

```
GBP: 42,00 EUR = 35,70 GBP (0,6 s)  
USD: 42,00 EUR = 45,36 USD (1,0 s)  
JPY: 42,00 EUR = 6636,00 JPY (1,2 s)  
Alle Umrechnungen abgeschlossen (1,2 s)
```

Parallel Stream

1. Erstelle eine Liste mit folgenden Namen:
`"Paul", "Vladimir", "Jessica", "Duncan", "Chani", "Leto"`
2. Implementiere Folgendes einmal mit einem seriellen und einmal mit einem parallelen Stream:
 1. Es sollen nur Namen bearbeitet werden, deren **Buchstabenanzahl größer gleich 5** ist.
 2. Wandle diese Namen in **Großbuchstaben** um.
 3. **Schlafe** vor jeder Umwandlung für **100 ms**.
 4. Gebe alle umgewandelten Namen schließlich auf der Konsole aus.
3. Berechne die Laufzeit mit **java.time.Instant** für beide Varianten.
Gebe die Dauer für beide Varianten in Millisekunden auf der Konsole aus.

Ausgabe:

```
Serial:    [VLADIMIR, JESSICA, DUNCAN, CHANI]  
Duration: 438 ms  
Parallel: [VLADIMIR, JESSICA, DUNCAN, CHANI]  
Duration: 106 ms
```

Reduce

Berechne **10 + die Summe aus folgenden Zahlen**: [10, 20, 30, 40]. Verwende hierzu die Methode **Stream::reduce**.

Schreibe eine Lösung sowohl mit **Stream** als auch mit **Parallel-Stream**.

Gebe beide Ergebnisse in der Konsole aus.

Erwartetes Ergebnis:

110

JokeFetcher: StringParser

Lade das Projekt "**JokeFetcher**" aus **Moodle** herunter.

JokeFetcher ruft zufällige Jokes aus folgender API ab (siehe [Doku](https://official-joke-api.appspot.com/jokes/random)):
<https://official-joke-api.appspot.com/jokes/random>

Die Antworten liegen im **JSON-Format** vor. Beispiel:

```
{"type": "programming", "setup": "A SQL query walks into a bar, walks up to  
two tables and asks...", "punchline": "'Can I join you?'", "id": 24}
```

Implementiere zunächst die Klasse **JokeJsonStringParser**, sodass der zur Verfügung gestellte JUnit-Test **JokeJsonStringParserTest** deine Lösung akzeptiert.

Du darfst hierfür **nur Methoden aus der Klasse String verwenden**. Schaue in der Klasse [String](#) nach, welche Methode nützlich sein könnten und wie du die Methode **split** verwenden kannst.

Gehe bei deiner Implementierung davon aus, dass die Struktur des JSON immer gleich ist, d. h., es gibt immer ein type, ein setup, eine punchline, eine id und diese sind immer in gleicher Reihenfolge und es gibt immer Werte (kein null).

JokeFetcher: JacksonParser

Implementiere nun die Klasse **JokeJsonJacksonParser**, sodass der zur Verfügung gestellte JUnit-Test **JokeJsonJacksonParserTest** deine Lösung akzeptiert.

Nutze für diese Implementierung **Jackson**. Jackson ist eine weit verbreitete Java-Bibliothek, die zum Parsen von JSON verwendet wird. Die Dependency ist bereits in dem zur Verfügung gestellten Projekt enthalten.

Recherchiere im Internet, wie du Jackson verwenden kannst.

JokeFetcher: fetchRandomJokes

Implementiere schließlich die folgende Methode in der Klasse **JokeFetcher**:

```
/**
 * Requests a certain number of random jokes from a joke API.
 * All requests are made <b>asynchronous (in parallel)</b>.
 * The jokes are parsed from JSON to {@link Joke}.
 *
 * @param limit a positive number of jokes to be fetched.
 * @return a collection with fetched and parsed {@link Joke} objects.
 * @throws IllegalArgumentException if limit is below zero.
 * @see HttpClient#sendAsync
 * @see CompletableFuture
 */
Collection<Joke> fetchRandomJokes(int limit) {
    return new LinkedList<>(); // TODO
}
```

Die Methode soll eine bestimmte Anzahl (limit) an zufälligen Jokes aus der genannten Joke-API abrufen. Dies muss **parallel implementiert** werden: mehrere API-Anfragen sollen **gleichzeitig** gestellt werden. Die Ergebnisse müssen **geparst** werden (siehe **JokeJsonParser**) und werden als Collection zurückgegeben.

Prüfe deine Lösung mit dem zur Verfügung gestellten JUnit-Test **JokeFetcherTest**.